INDEX

S.No.	Date	Excercises	Page No.	Marks	sign
1.		Installing Data Analysis And Visualization			
2.		Exploratory Data Analysis (EDA) On With Datasets			
3.		Working With Numpy Arrays, Pandas Data Frames , Basic Plots Using Matplotlib			
4.		Exploring Various Variable And Row Filters In R For Cleaning Data			
5.		Perform Eda On Wine Quality Data Set.			
6.		Time Series Analysis Using Various Visulaization Techniques			
7.		Data Analysis And Representation On A Map			
8.		Building Cartographic Visualization			
9.		Use A Case Study On A Data Set And Visualization Technique And Present Analysis Report			
10		Advanced Interactive Dashboard And Real Time Data Visualization			

EX NO: 1	INSTALLING DATA ANALYSIS AND VISUALIZATION
DATE:	

AIM:

To write a steps to install data Analysis and Visualization tool: R/ Python /Tableau Public/ Power BI.

PROCEDURE:

R:

- R is a programming language and software environment specifically designed for statistical computing and graphics.

Windows:

- Download R from the official website: https://cran.r-project.org/mirrors.html
- Run the installer and follow the installation instructions.

macOS:

- Download R for macOS from the official website: https://cran.r-project.org/mirrors.html
- Open the downloaded file and follow the installation instructions.

Linux:

- You can typically install R using your distribution's package manager. For example, on Ubuntu, you can use the following command:

csharp

Copy code

sudo apt-get install r-base

Python:

- Python is a versatile programming language widely used for data analysis. You can install Python and data analysis libraries using a package manager like conda or pip.

Windows:

- Download Python from the official website: https://www.python.org/downloads/windows/
- Run the installer, and make sure to check the "Add Python to PATH" option during installation.
- You can install data analysis libraries like NumPy, pandas, and matplotlib using pip.

macOS:

- macOS typically comes with Python pre-installed. You can install additional packages using pip or set up a virtual environment using Ana

- conda.

Linux:

- Python is often pre-installed on Linux. Use your distribution's package manager to install Python if it's not already installed. You can also use conda or pip to manage Python package

Tableau Public:

- Tableau Public is a free version of Tableau for creating and sharing interactive data visualizations.
- Go to the Tableau Public website: https://public.tableau.com/s/gallery
- Download and install Tableau Public by following the instructions on the website.

Power BI:

- Power BI is a business analytics service by Microsoft for creating interactive reports and dashboards.
- Go to the Power BI website: https://powerbi.microsoft.com/en-us/downloads/
- Download and install Power BI Desktop, which is the tool for creating reports and dashboards.
- Please note that the installation steps may change over time, so it's a good idea to check the official websites for the most up-to-date instructions and download links. Additionally, system requirements may vary, so make sure your computer meets the necessary specifications.

Result:

Thus the python program to install data analysis and visualization tools like R,Python,Tableau public or power BI , and their features were explored successfully

EX NO: 2	Exploratory Data Analysis (EDA) on with Datasets
DATE:	

Aim:

To Perform exploratory data analysis (EDA) on with datasets like email data set.

Procedure:

Exploratory Data Analysis (EDA) on email datasets involves importing the data, cleaning it, visualizing it, and extracting insights. Here's a step-by-step guide on how to perform EDA on an email dataset using Python and Pandas

1. **Import Necessary Libraries**:

Import the required Python libraries for data analysis and visualization.

2. **Load Email Data:**

Assuming you have a folder containing email files (e.g., .eml files), you can use the email library to parse and extract the email contents.

3. **Data Cleaning:**

Depending on your dataset, you may need to clean and preprocess the data. Common cleaning steps include handling missing values, converting dates to datetime format, and removing duplicates.

4. **Data Exploration**:

Now, you can start exploring the dataset using various techniques. Here are some common EDA tasks:

Basic Statistics:

Get summary statistics of the dataset.

Distribution of Dates:

Visualize the distribution of email dates.

5. **Word Cloud for Subject or Message**:

Create a word cloud to visualize common words in email subjects or messages.

6. **Top Senders and Recipients:**

Find the top email senders and recipients.

Depending on your dataset, you can explore further, analyze sentiment, perform network analysis, or any other relevant analysis to gain insights from your email data.

Program:

```
# Import necessary libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
# Load the dataset
df = pd.read_csv('D:\ARCHANA\dxv\LAB\DXV\Emaildataset.csv')
# Display basic information about the dataset
print(df.info())
# Display the first few rows of the dataset
print(df.head())
# Descriptive statistics
print(df.describe())
# Check for missing values
print(df.isnull().sum())
# Visualize the distribution of numerical variables
sns.pairplot(df)
plt.show()
# Visualize the distribution of categorical variables
sns.countplot(x='label', data=df)
plt.show()
# Correlation matrix for numerical variables
correlation_matrix = df.corr()
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.show()
# Word cloud for text data (if you have a column with text data)
from wordcloud import WordCloud
text_data = ' '.join(df['text_column'])
wordcloud = WordCloud(width=800, height=400, random_state=21,
max_font_size=110).generate(text_data)
plt.figure(figsize=(10, 7))
plt.imshow(wordcloud, interpolation="bilinear")
plt.axis('off')
```

plt.show()

OUT PUT:

Data columns (total 4 columns):

Column Non-Null Count Dtype

--- ------ -------------- -----

0 Unnamed: 0 5171 non-null int64

1 label 5171 non-null object

2 text 5171 non-null object

3 label_num 5171 non-null int64

dtypes: int64(2), object(2)

memory usage: 161.7+ KB

None

Unnamed: 0 label text label_num

0 605 ham Subject: enron methanol ; meter # : 988291\r\n... 0

1 2349 ham Subject: hpl nom for january 9 , 2001\r\n(see... 0

2 3624 ham Subject: neon retreat\r\nho ho ho , we ' re ar... 0

3 4685 spam Subject: photoshop , windows , office . cheap ... 1

4 2030 ham Subject: re : indian springs\r\nthis deal is t... 0

Unnamed: 0 label_num

count 5171.000000 5171.000000

mean 2585.000000 0.289886

std 1492.883452 0.453753

min 0.000000 0.000000

25% 1292.500000 0.000000

50% 2585.000000 0.000000

75% 3877.500000 1.000000

max 5170.000000 1.000000

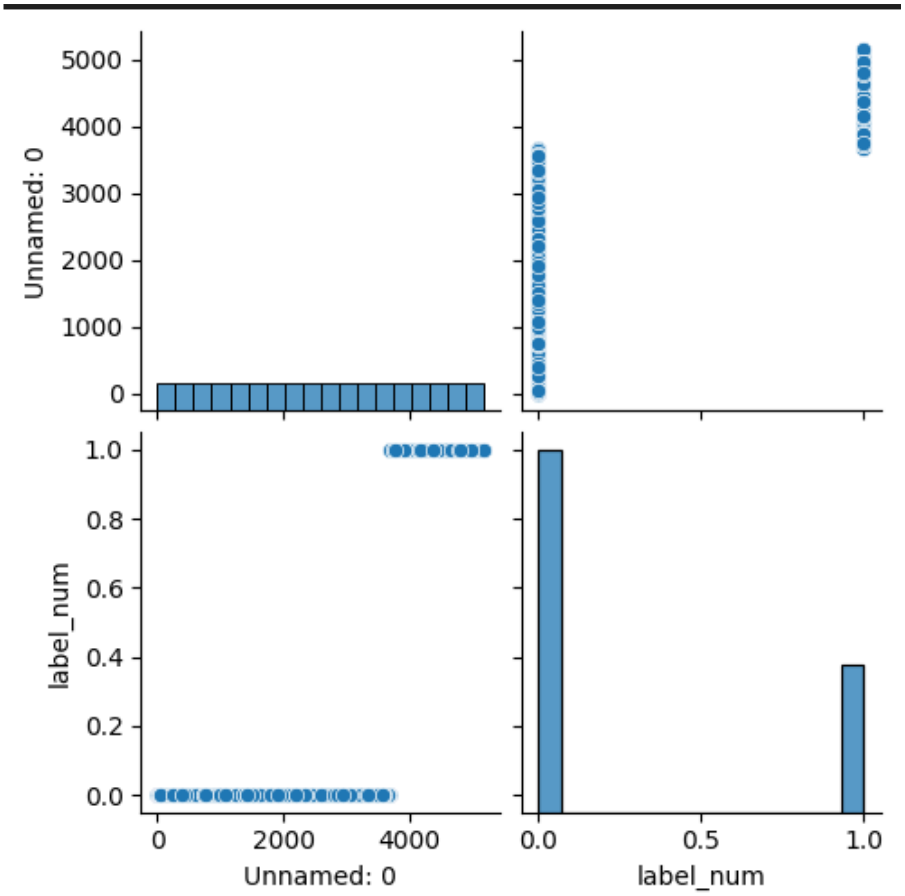
Unnamed: 0 0

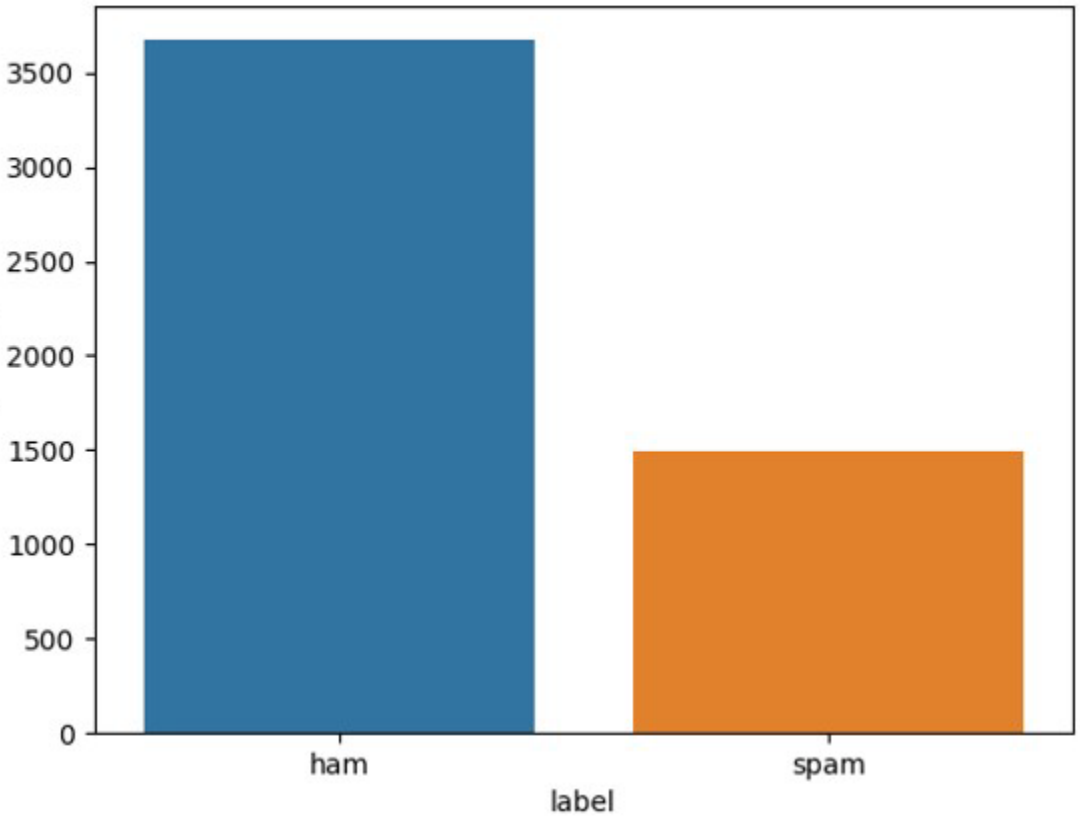
label 0

text 0

label_num 0

dtype: int64


Unnamed: 0
label_num
Unnamed: 0
label_num


label
ham
spam

Result:

The above Performing exploratory data analysis (EDA) on with datasets like email data set has been performed successfully.

EX NO: 3	Working with Numpy arrays, Pandas data frames , Basic plots using Matplotlib
DATE:	

Aim:

Write the steps for Working with Numpy arrays, Pandas data frames , Basic plots using Matplotlib

Procedure:

1. NumPy:

NumPy is a fundamental library for numerical computing in Python. It provides support for multidimensional arrays and various mathematical functions. To get started, you'll first need to install NumPy if you haven't already (you can use pip):

Once NumPy is installed, you can use it as follows:

```
import numpy as np
# Creating NumPy arrays
arr = np.array([1, 2, 3, 4, 5])
print(arr)
# Basic operations
mean = np.mean(arr)
sum = np.sum(arr)
# Mathematical functions
square_root = np.sqrt(arr)
exponential = np.exp(arr)
# Indexing and slicing
first_element = arr[0]
sub_array = arr[1:4]
# Array operations
combined_array = np.concatenate([arr, sub_array])
```

OUTPUT:



2. Pandas:

Pandas is a powerful library for data manipulation and analysis.

You can install Pandas using pip:

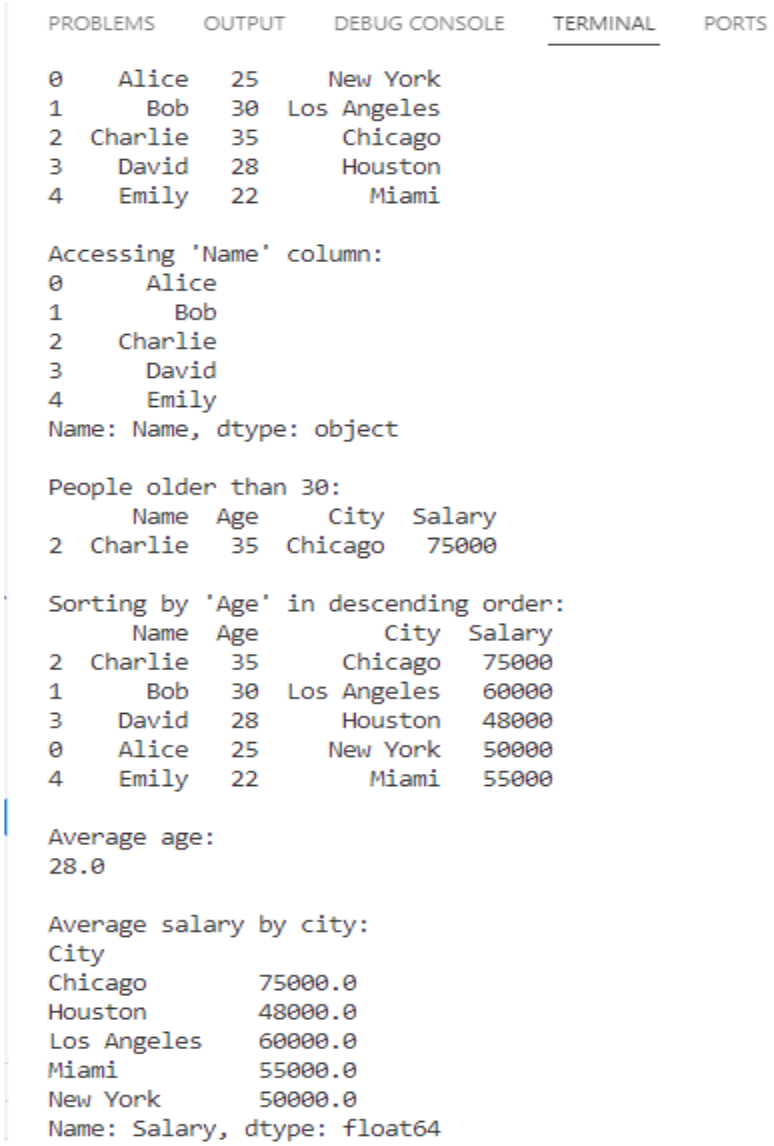
pip install pandas

Here's how to work with Pandas DataFrames:

```
import pandas as pd
# Creating a DataFrame from a dictionary
data = {
'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Emily'],
'Age': [25, 30, 35, 28, 22],
'City': ['New York', 'Los Angeles', 'Chicago', 'Houston', 'Miami']
}
df = pd.DataFrame(data)
# Display the entire DataFrame
print("DataFrame:")
print(df)
# Accessing specific columns
print("\nAccessing 'Name' column:")
print(df['Name'])
# Adding a new column
df['Salary'] = [50000, 60000, 75000, 48000, 55000]
# Filtering data
print("\nPeople older than 30:")
print(df[df['Age'] > 30])
# Sorting by a column
print("\nSorting by 'Age' in descending order:")
print(df.sort_values(by='Age', ascending=False))
# Aggregating data
print("\nAverage age:")
print(df['Age'].mean())
# Grouping and aggregation
grouped_data = df.groupby('City')['Salary'].mean()
print("\nAverage salary by city:")
print(grouped_data)
```

```
# Applying a function to a column
df['Age_Squared'] = df['Age'].apply(lambda x: x ** 2)
# Removing a column
df = df.drop(columns=['Age_Squared'])
# Saving the DataFrame to a CSV file
df.to_csv('output.csv', index=False)
# Reading a CSV file into a DataFrame
new_df = pd.read_csv('output.csv')
print("\nDataFrame from CSV file:")
print(new_df)
```

OUTPUT:



3. Matplotlib:

Matplotlib is a popular library for creating static, animated, or interactive plots and graphs.

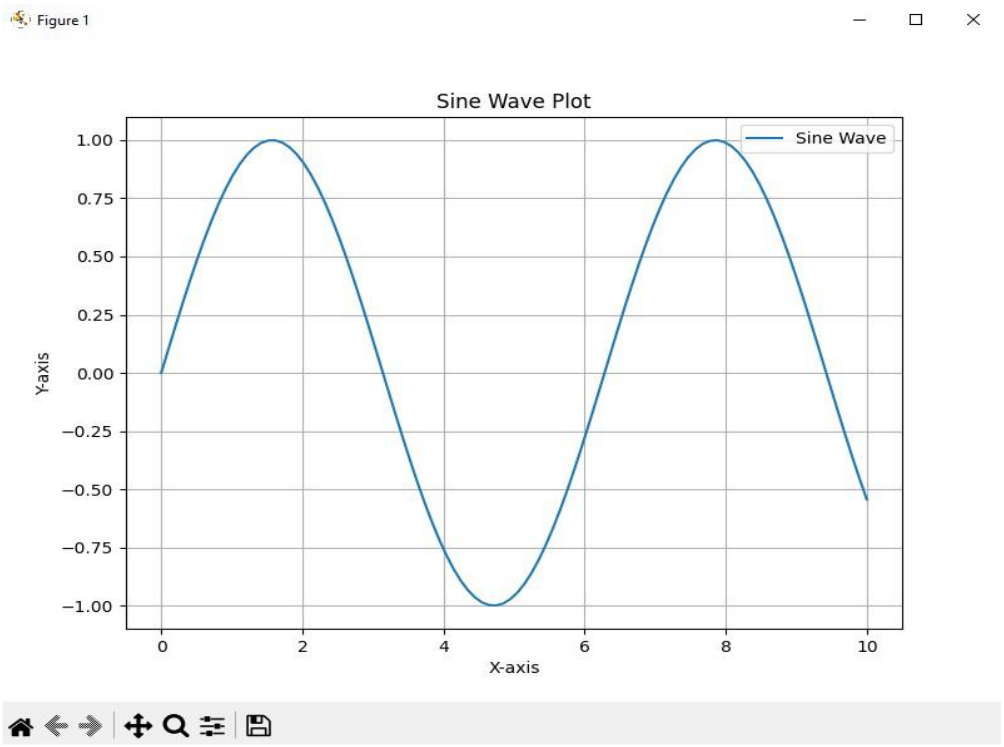
Install Matplotlib using pip:

pip install matplotlib

Here's a simple example of creating a basic plot:

```
import matplotlib.pyplot as plt
# Sample data
x = np.linspace(0, 10, 100)
y = np.sin(x)
# Create a line plot
plt.figure(figsize=(8, 6))
plt.plot(x, y, label='Sine Wave')
plt.title('Sine Wave Plot')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.legend()
plt.grid(True)
plt.show()
```

OUTPUT:



RESULT:

Thus the above working with numpy, pandas, matplotlib has been completed successfully.

EX NO: 4	Exploring various variable and row filters in R for cleaning data
DATE:	

Aim:

Exploring various variable and row filters in R for cleaning data.

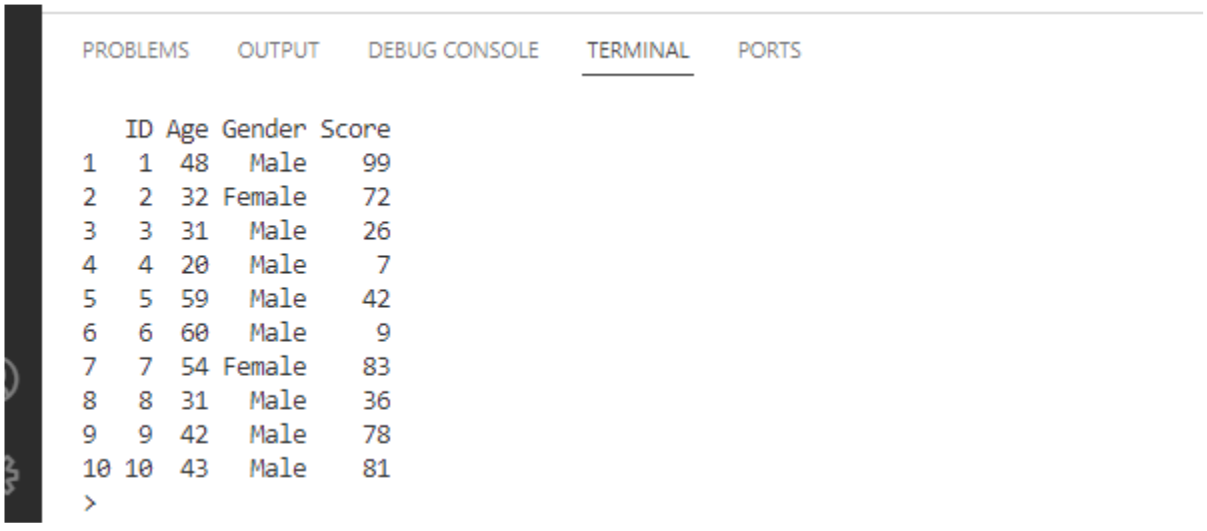
PROCEDURE:

Data Preparation and Cleaning

First, let's create a sample dataset and then explore various variable and row filters to clean the data

```
# Create a sample dataset
set.seed(123)
data <- data.frame(
ID = 1:10,
Age = sample(18:60, 10, replace = TRUE),
Gender = sample(c("Male", "Female"), 10, replace = TRUE),
Score = sample(1:100, 10)
)
# Print the sample data
print(data)
```

OUTPUT:



Variable Filters

1. Filtering by a Specific Value:

To filter rows based on a specific value in a variable (e.g., only show rows where Age is greater than 30):

```
filtered_data <- data[data$Age > 30, ]
```

2. Filtering by Multiple Conditions:

- You can filter rows based on multiple conditions using the & (AND) or | (OR) operators (e.g., show rows where Age is greater than 30 and Gender is "Male"):

```
filtered_data <- data[data$Age > 30 & data$Gender == "Male", ]
```

Row Filters

1. Removing Duplicate Rows:

To remove duplicate rows based on certain columns (e.g., remove duplicates based on 'ID'):

```
cleaned_data <- unique(data[, c("ID", "Age", "Gender")])
```

2. Removing Rows with Missing Values:

To remove rows with missing values (NA):

```
cleaned_data <- na.omit(data)
```

Data Visualization

1. Apply various plot features using the ggplot2 package to visualize the cleaned data.

```
# Load the ggplot2 package
library(ggplot2)
# Create a scatterplot of Age vs. Score with points colored by Gender
ggplot(data = cleaned_data, aes(x = Age, y = Score, color = Gender)) +
geom_point() +
labs(title = "Scatterplot of Age vs. Score",
x = "Age",
y = "Score")
# Create a histogram of Age
ggplot(data = cleaned_data, aes(x = Age)) +
geom_histogram(binwidth = 5, fill = "blue", alpha = 0.5) +
labs(title = "Histogram of Age",
x = "Age",
y = "Frequency")
# Create a bar chart of Gender distribution
ggplot(data = cleaned_data, aes(x = Gender)) +
```

```
geom_bar(fill = "green", alpha = 0.7) +
labs(title = "Gender Distribution",
x = "Gender",
y = "Count")
```

RESULT:

Thus the above Exploring various variable and row filters in R for cleaning data.

EX NO: 5	PERFORM EDA ON WINE QUALITY DATA SET.
DATE:	

AIM:

To write a program to Perform EDA on Wine Quality Data Set.

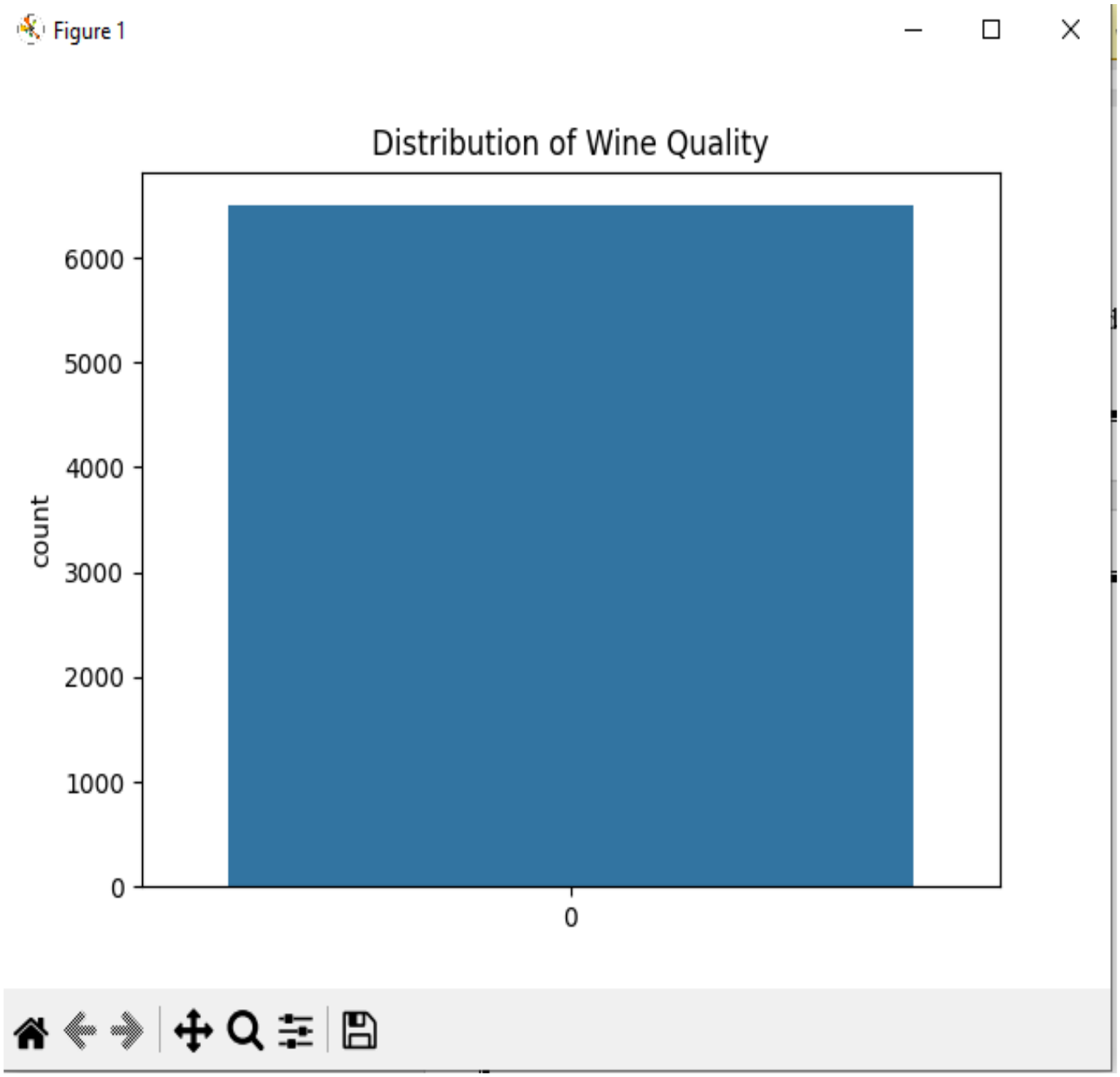
PROGRAM:

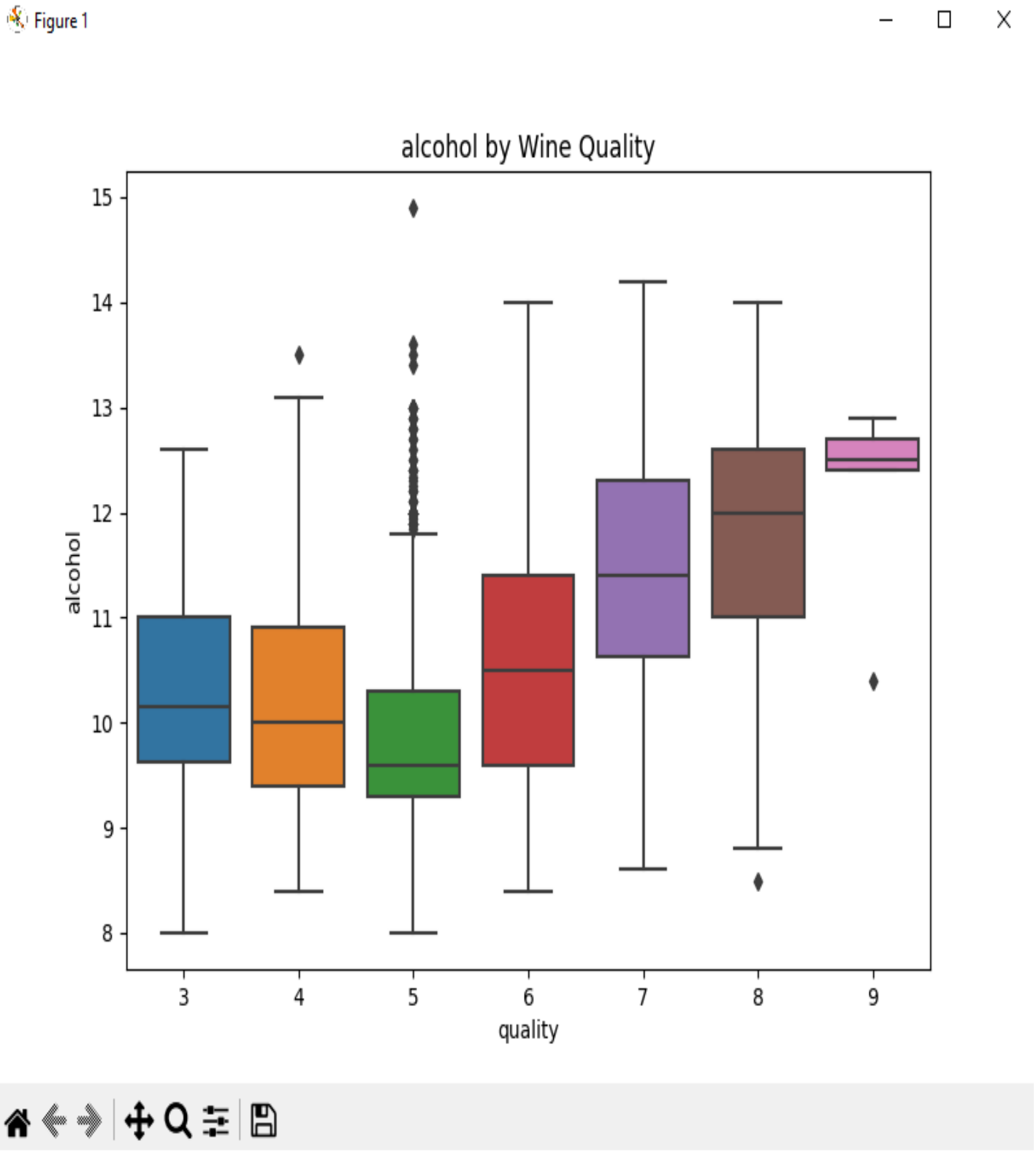
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
# Load the dataset
data = pd.read_csv("pathname")
# Display the first few rows of the dataset
print(data.head())
# Get information about the dataset
print(data.info())
# Summary statistics
print(data.describe())
# Distribution of wine quality
sns.countplot(data['quality'])
plt.title(" Wine Quality data set")
plt.show()
# Box plots for selected features by wine quality
features = ['alcohol', 'volatile acidity', 'citric acid', 'residual sugar']
for feature in features:
plt.figure(figsize=(8, 6))
sns.boxplot(x='quality', y=feature, data=data)
plt.title(f'{feature} by Wine Quality')
plt.show()
# Pair plot of selected features
sns.pairplot(data, vars=['alcohol', 'volatile acidity', 'citric acid', 'residual sugar'],
hue='quality', diag_kind='kde')
plt.suptitle("Pair Plot of Selected Features")
plt.show()
```

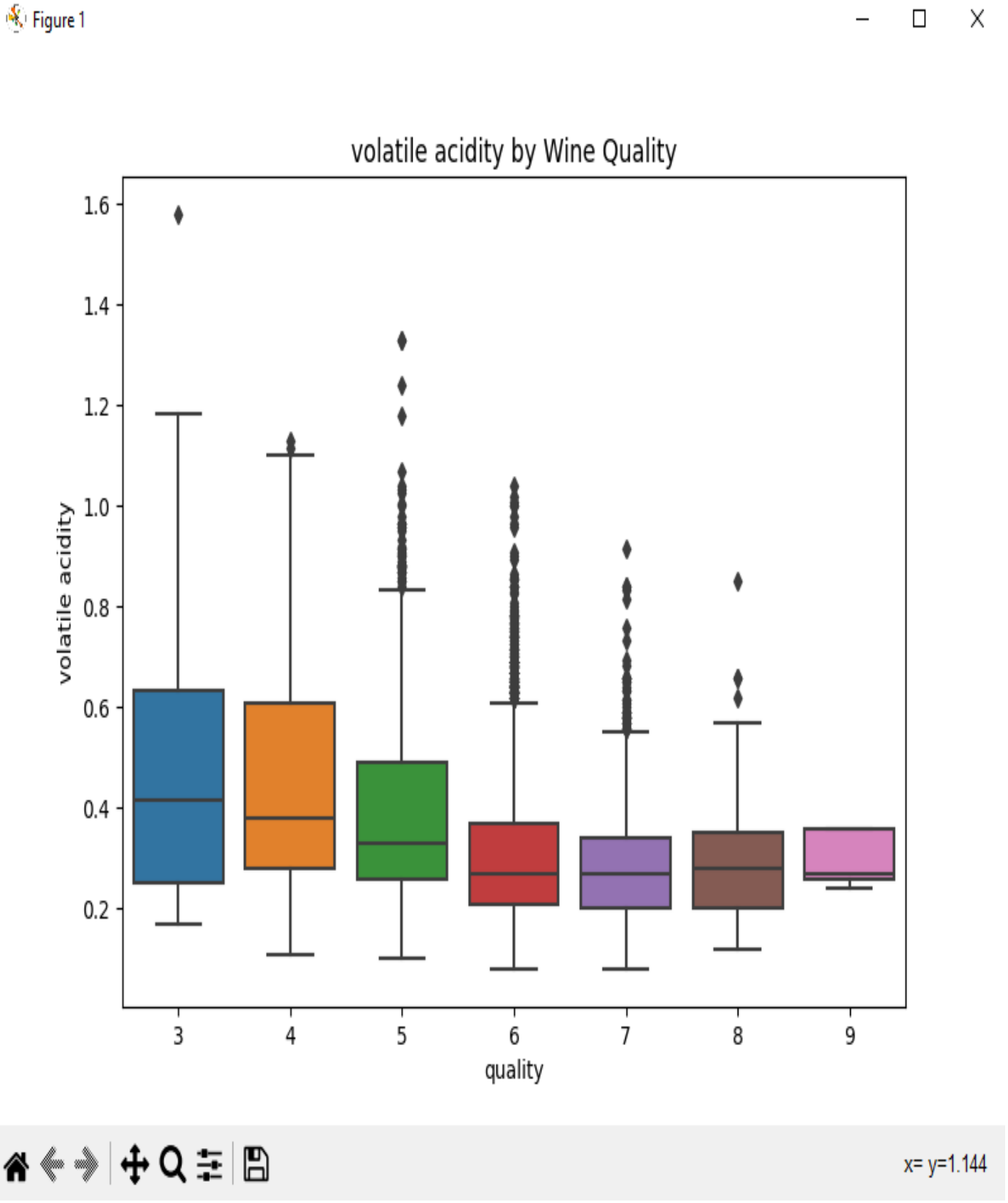
```
# Correlation heatmap
corr_matrix = data.corr()
plt.figure(figsize=(10, 8))
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Heatmap")
plt.show()
# Histograms of selected features
features = ['alcohol', 'volatile acidity', 'citric acid', 'residual sugar']
for feature in features:
plt.figure(figsize=(6, 4))
sns.histplot(data[feature], kde=True, bins=20)
plt.title(f"Distribution of {feature}")
plt.show()
```

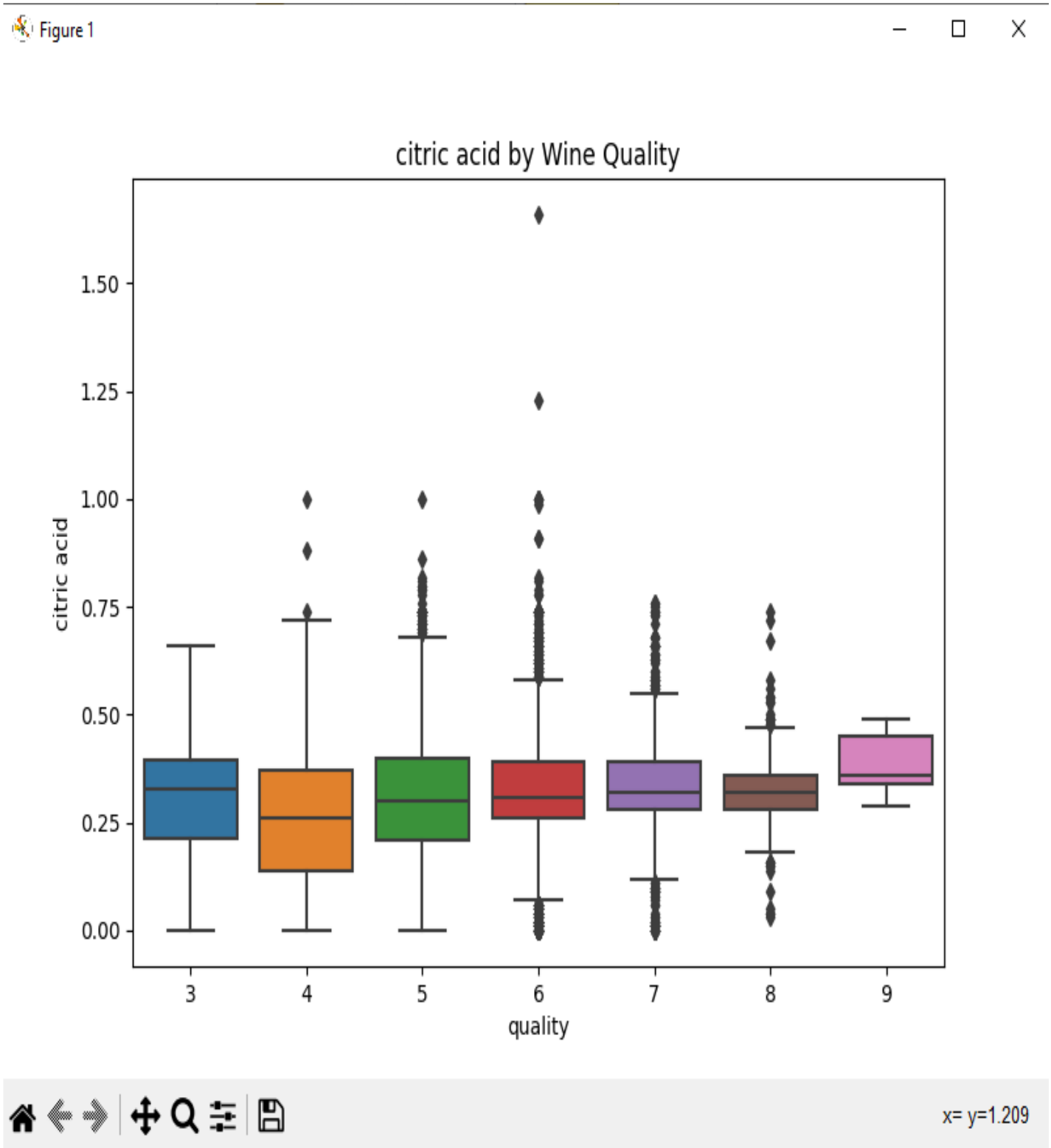
OUTPUT:

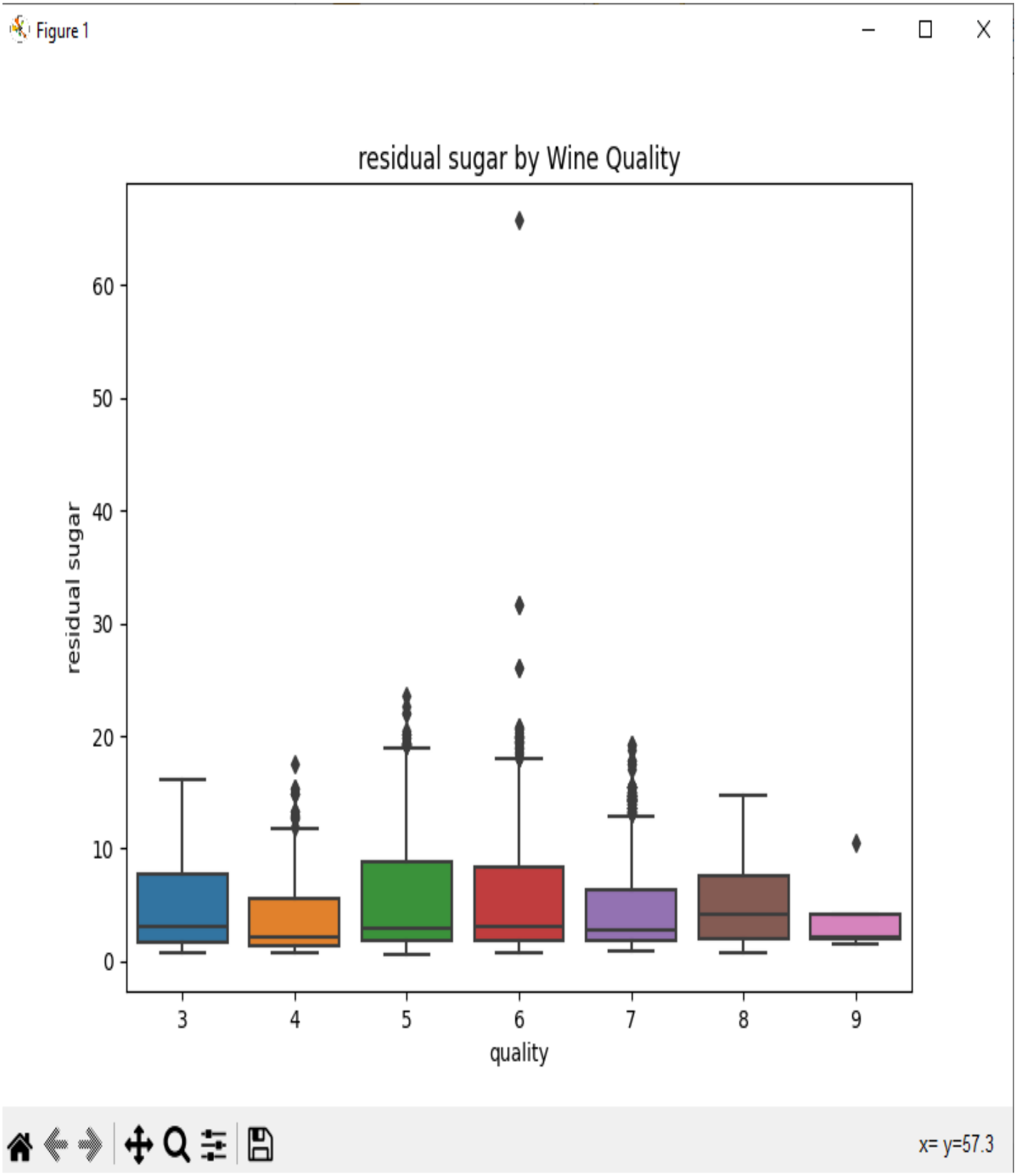
```
PS D:\ARCHANA\dxv\LAB> & C:/Users/ADMIN/AppData/Local/Microsoft/WindowsApps/python3.11.exe d:/ARCHANA/dxv/LAB/DXV/exno6.py
    type  alcohol  volatile acidity  citric acid  residual sugar  chlorides  free sulfur dioxide  total sulfur dioxide  density    pH  sulphates  quality  fixed acidity
0  white      8.8              0.27         0.36            20.7      0.045                 45.0                 170.0   1.0010  3.00       0.45        6            7.0
1  white      9.5              0.30         0.34             1.6      0.049                 14.0                 132.0   0.9940  3.30       0.49        6            6.3
2  white     10.1              0.28         0.40             6.9      0.050                 30.0                  97.0   0.9951  3.26       0.44        6            8.1
3  white      9.9              0.23         0.32             8.5      0.058                 47.0                 186.0   0.9956  3.19       0.40        6            7.2
4  white      9.9              0.23         0.32             8.5      0.058                 47.0                 186.0   0.9956  3.19       0.40        6            7.2
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6497 entries, 0 to 6496
Data columns (total 13 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   type                  6497 non-null   object
 1   alcohol               6497 non-null   float64
 2   volatile acidity      6489 non-null   float64
 3   citric acid           6494 non-null   float64
 4   residual sugar        6495 non-null   float64
 5   chlorides             6495 non-null   float64
 6   free sulfur dioxide   6497 non-null   float64
 7   total sulfur dioxide  6497 non-null   float64
 8   density               6497 non-null   float64
 9   pH                    6488 non-null   float64
 10  sulphates             6493 non-null   float64
 11  quality               6497 non-null   int64
 12  fixed acidity         6487 non-null   float64
dtypes: float64(11), int64(1), object(1)
memory usage: 660.0+ KB
None
           alcohol  volatile acidity  citric acid  residual sugar    chlorides  free sulfur dioxide  total sulfur dioxide      density           pH    sulphates      quality  fixed acidity
count  6497.000000       6489.000000  6494.000000     6495.000000  6495.000000          6497.000000           6497.000000  6497.000000  6488.000000  6493.000000  6497.000000    6487.000000
mean     10.491801          0.339691     0.318722        5.444326     0.056042            30.525319            115.744574     0.994697     3.218395     0.531215     5.818378       7.216579
std       1.192712          0.164649     0.145265        4.758125     0.035036            17.749400             56.521855     0.002999     0.160748     0.148814     0.873255       1.296750
min       8.000000          0.080000     0.000000        0.600000     0.009000             1.000000              6.000000     0.987110     2.720000     0.220000     3.000000       3.800000
25%       9.500000          0.230000     0.250000        1.800000     0.038000            17.000000             77.000000     0.992340     3.110000     0.430000     5.000000       6.400000
50%      10.300000          0.290000     0.310000        3.000000     0.047000            29.000000            118.000000     0.994890     3.210000     0.510000     6.000000       7.000000
75%      11.300000          0.400000     0.390000        8.100000     0.065000            41.000000            156.000000     0.996990     3.320000     0.600000     6.000000       7.700000
max      14.900000          1.580000     1.660000       65.800000     0.611000           289.000000            440.000000     1.038980     4.010000     2.000000     9.000000      15.900000
```

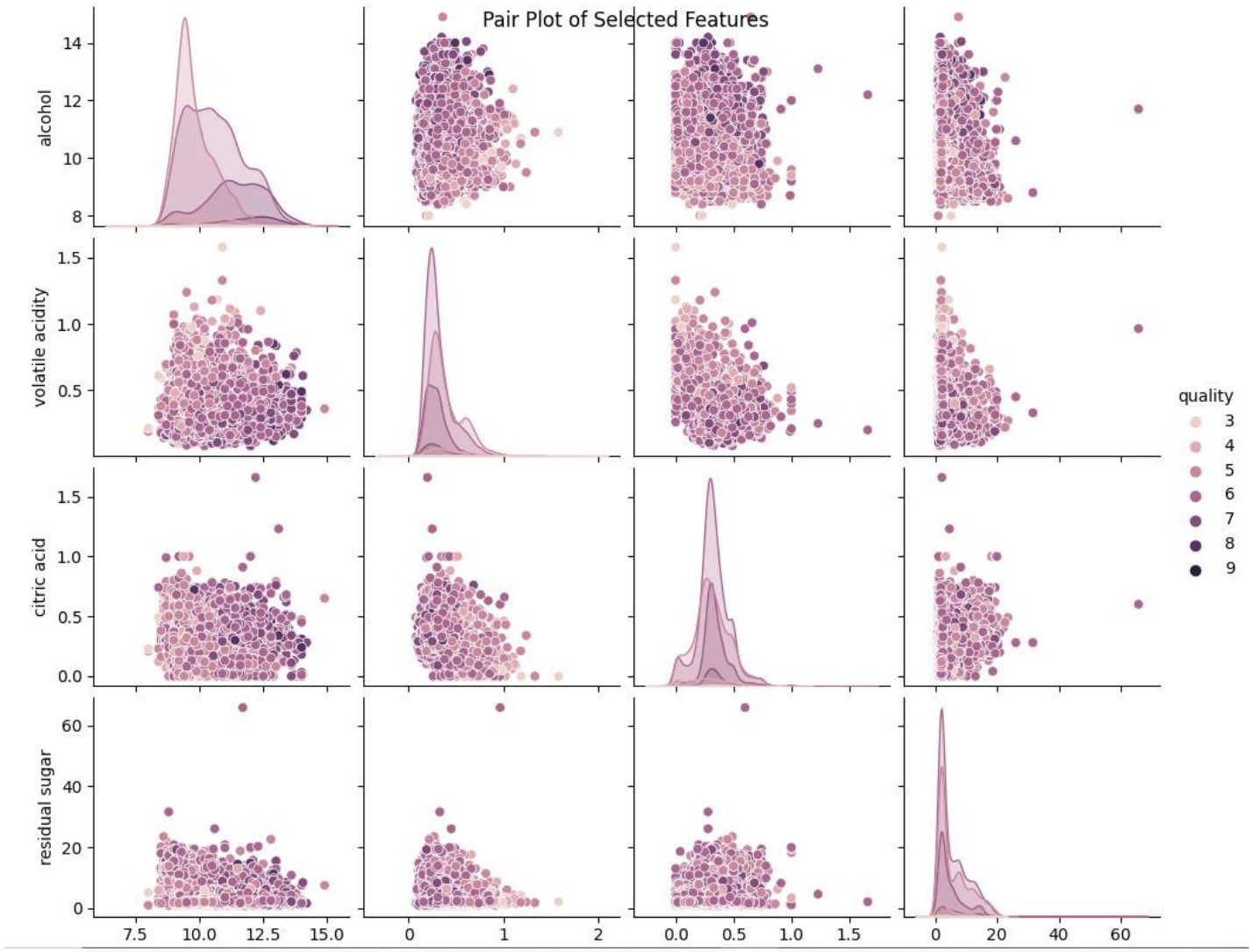

Figure 1
Distribution of Wine Quality
count
6000
5000
4000
3000
2000
1000
0
0


Figure 1
alcohol by Wine Quality
alcohol
quality
15
14
13
12
11
10
9
8
3
4
5
6
7
8
9


Figure 1
volatile acidity by Wine Quality
volatile acidity
1.6
1.4
1.2
1.0
0.8
0.6
0.4
0.2
3
4
5
6
7
8
9
quality
x= y=1.144


Figure 1
citric acid by Wine Quality
citric acid
1.50
1.25
1.00
0.75
0.50
0.25
0.00
3
4
5
6
7
8
9
quality
x= y=1.209


Figure 1
residual sugar by Wine Quality
residual sugar
0
10
20
30
40
50
60
3
4
5
6
7
8
9
quality
x= y=57.3



RESULT:

Thus the above program to to Perform EDA on Wine Quality Data Set.

EX NO: 6	TIME SERIES ANALYSIS USING VARIOUS VISULAIZATION TECHNIQUES
DATE:	

AIM:

To perform time series analysis and apply the various visualization techniques.

DOWNLOADING DATASET:

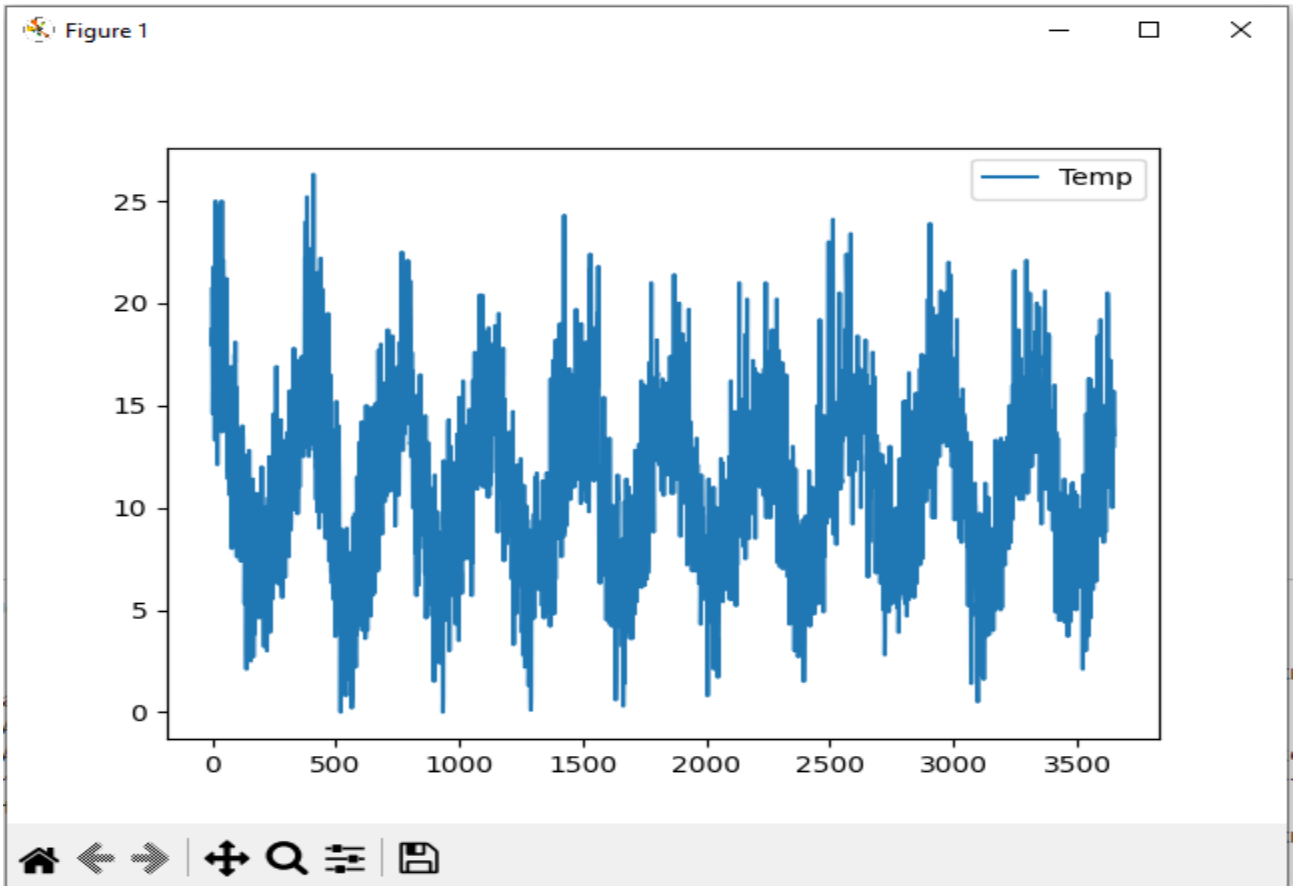
Step 1: Open google and type the following path in the address bar and download a dataset.

http: //github.com/jbrownlee/Datasets .

Step 2: write the following code to get the details.

```
from pandas import read_csv
from matplotlib import pyplot
series=read_csv('pathname')
print(series.head())
series.plot()
pyplot.show()
```

OUTPUT:

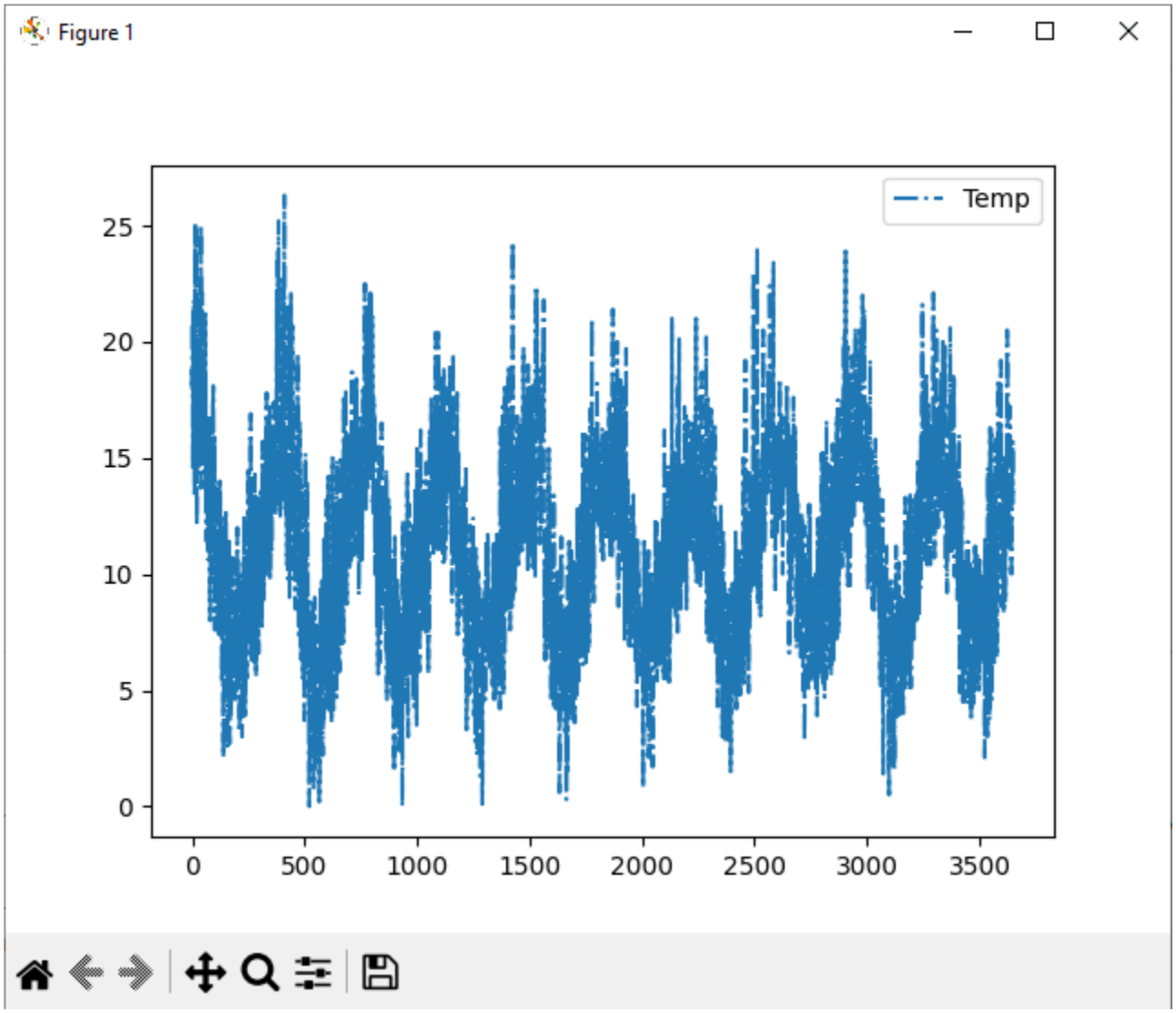


```
PS D:\ARCHANA\dxv\LAB> & C:/Users/ADMIN/AppData/Local/Microsoft/WindowsApps/python3.11.exe d:/ARCHANA/dxv/LAB/DXV/exno5new.py
         Date  Temp
0  01-01-1981  20.7
1  02-01-1981  17.9
2  03-01-1981  18.8
3  04-01-1981  14.6
4  05-01-1981  15.8
```

Step 3: To get the time series line plot:

series.plot(style='-.')

pyplot.show()



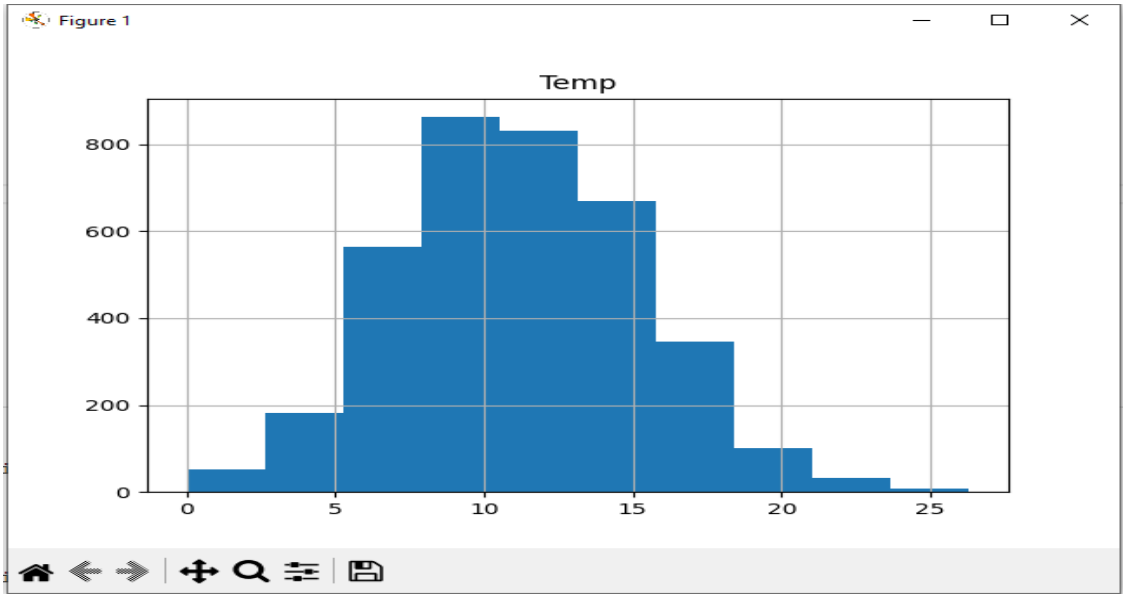

```
PS D:\ARCHANA\dxv\LAB> & C:/Users/ADMIN/AppData/Local/Microsoft/WindowsApps/python3.11.exe d:/ARCHANA/dxv/LAB/DXV/exno5new.py
         Date  Temp
0  01-01-1981  20.7
1  02-01-1981  17.9
2  03-01-1981  18.8
3  04-01-1981  14.6
4  05-01-1981  15.8
```

Step 4:

To create a Histogram:

series.hist()

pyplot.show()

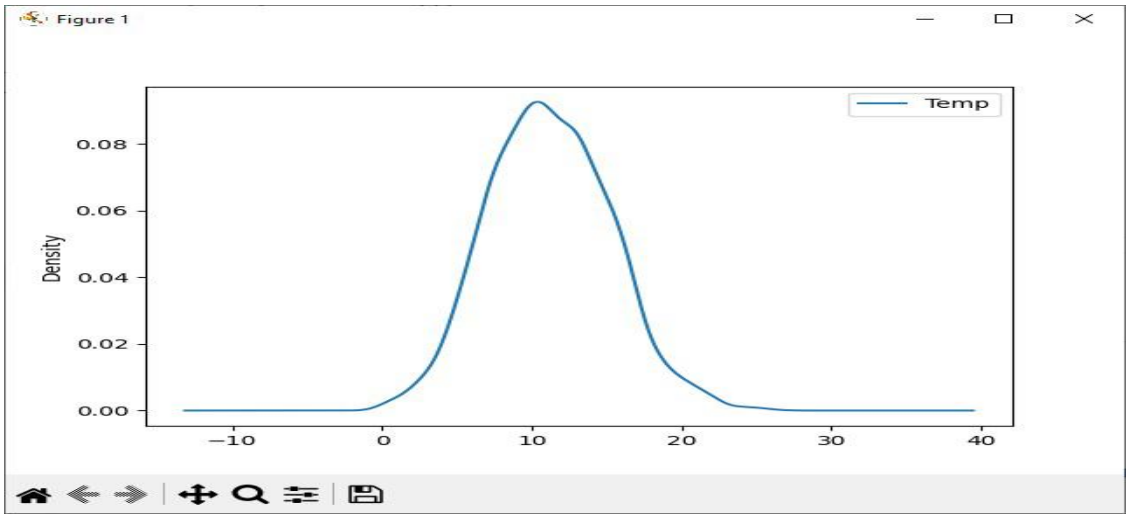


Step 5:

To create density plot:

series.plot(kind='kde')

pyplot.show()



Result:

Thus the above time analysis has been checked with various visualization techniques.

EX NO: 7	DATA ANALYSIS AND REPRESENTATION ON A MAP
DATE:	

AIM:

Write a program to perform data analysis and representation on a map using various map data sets with mouse rollover effect, user interaction.

PROCEDURE:

STEP 1:

- Make sure to install the necessary libraries.

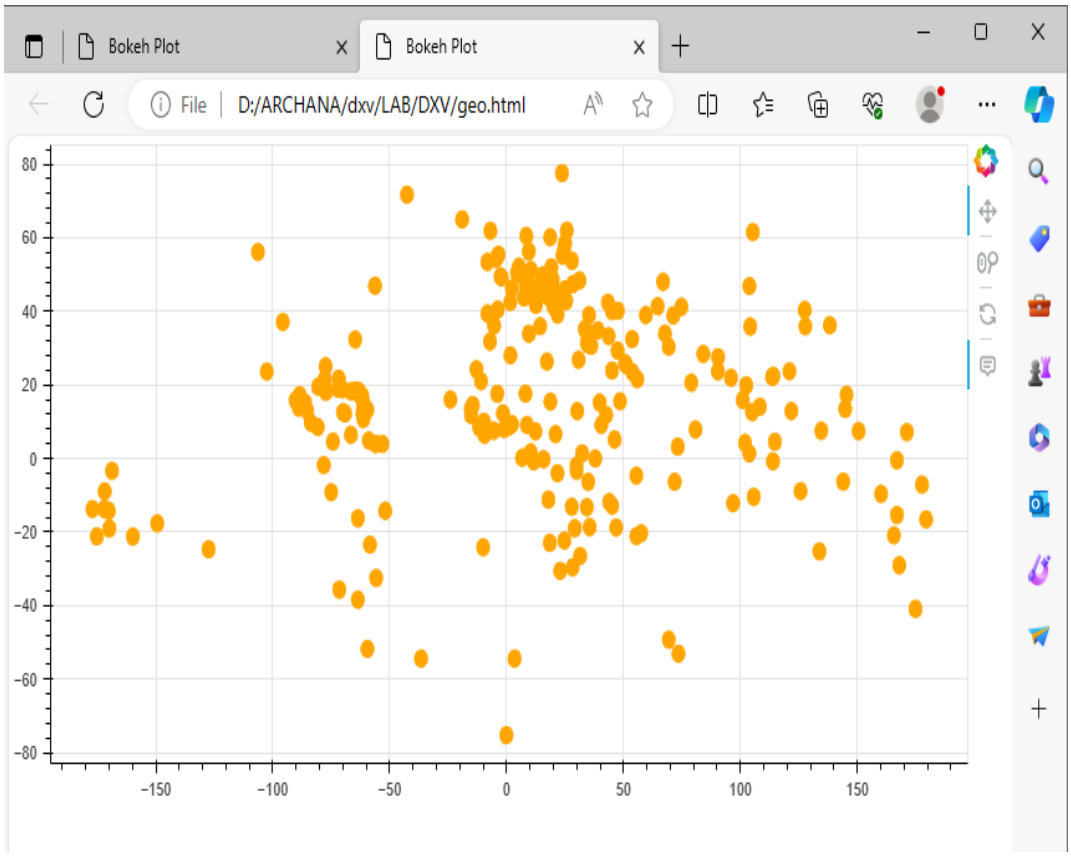
  pip install geopandas folium bokeh

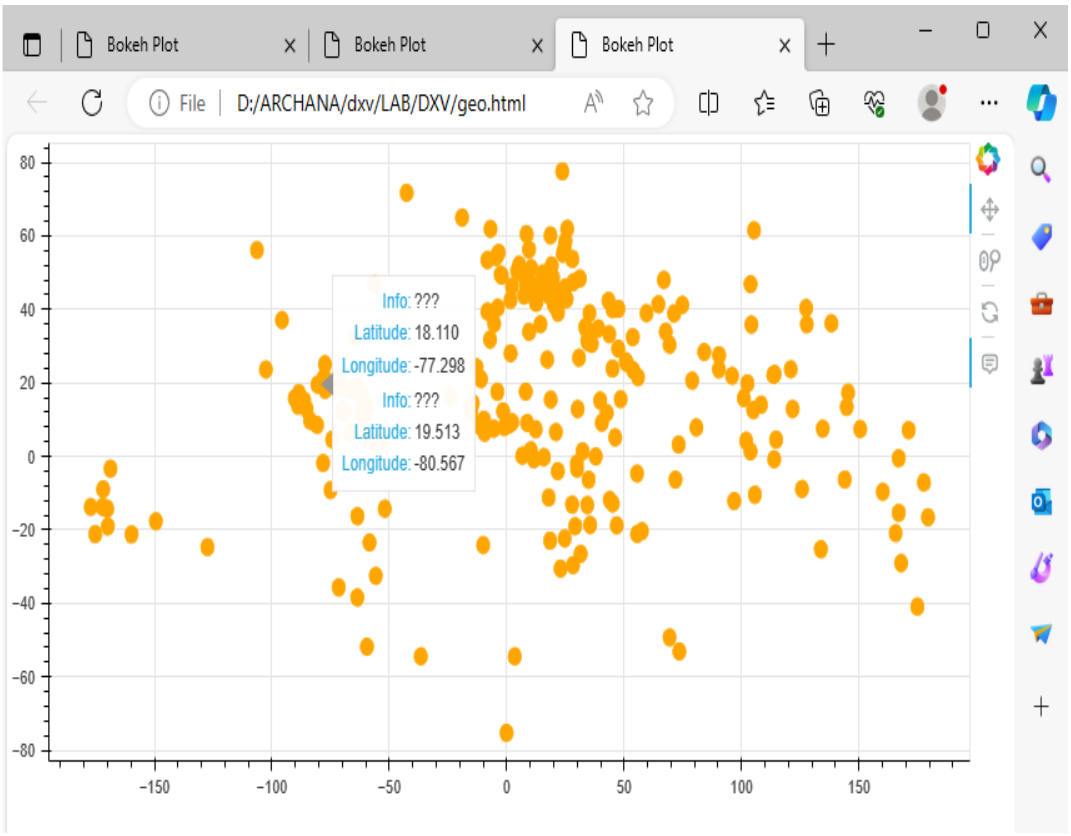
PROGRAM:

```
from bokeh.io import show
from bokeh.models import ColumnDataSource, HoverTool
from bokeh.plotting import figure
from bokeh.layouts import column
import pandas as pd
import folium
# Load your data
data = pd.read_csv('D:\ARCHANA\dxv\LAB\DXV\geographic.csv')
# Create a Bokeh figure
p = figure(width=800, height=400, tools='pan,wheel_zoom,reset')
# Create a ColumnDataSource to hold data
source = ColumnDataSource(data)
# Add circle markers to the figure
p.circle(x='Longitude', y='Latitude', size=10, source=source, color='orange')
# Create a hover tool for mouse rollover effect
hover = HoverTool()
hover.tooltips = [("Info", "@Info"), ("Latitude", "@Latitude"), ("Longitude",
"@Longitude")]
p.add_tools(hover)
# Display the Bokeh plot
layout = column(p)
```

```
show(layout)
# Create a map centered at a specific location
m = folium.Map(location=[latitude, longitude], zoom_start=10)
# Add markers for your data points
for index, row in data.iterrows():
folium.Marker(
location=[row['Latitude'], row['Longitude']],
popup=row['Info'], # Display additional info on mouse click
).add_to(m)
# Save the map to an HTML file
m.save('map.html')
```

OUTPUT:





RESULT:

Data analysis and representation on a map using various map data sets with mouse rollover effect, user interaction has been completed successfully.

EX NO: 8	BUILDING CARTOGRAPHIC VISUALIZATION
DATE:	

AIM:

Build cartographic visualization for multiple datasets involving various countries of the world; states and districts in India etc

PROCEDURE:

STEP 1:

Collect Datasets

Gather the datasets containing geographical information for countries, states, or districts. Make sure these datasets include the necessary attributes for mapping (e.g., country/state/district names, codes, and

relevant data).

STEP 2:

Install Required Libraries:

pip install geopandas matplotlib

STEP 3:

Load Geographic Data:

Use Geopandas to load the geographic data for countries, states, or districts. Make sure to match the geographical data with your datasets based on the common attributes.

STEP 4:

Merge Datasets:

Merge your datasets with the geographic data based on common attributes. This step is crucial for linking

your data to the corresponding geographic regions.

STEP 5:

Create Cartographic Visualizations:

Use Matplotlib to create cartographic visualizations. You can create separate plots for different datasets

or overlay them on a single map.

STEP 6:

Customize and Enhance:

Customize your visualizations based on your needs. You can add legends, labels, titles, and other elements to enhance the interpretability of your maps.

STEP 7:

Save and Share:

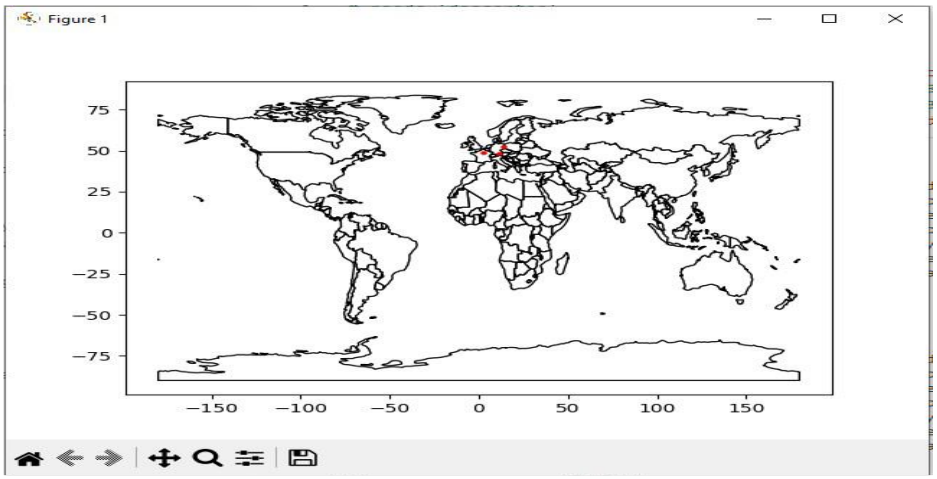
Save your visualizations as image files or interactive plots if needed. You can then share these visualizations with others.

PROGRAM:

```
import pandas as pd
import geopandas as gpd
import shapely
# needs 'descartes'
import matplotlib.pyplot as plt
df = pd.DataFrame({'city': ['Berlin', 'Paris', 'Munich'],
'latitude': [52.518611111111, 48.856666666667, 48.137222222222],
'longitude': [13.408333333333, 2.3516666666667, 11.575555555556]})
gdf = gpd.GeoDataFrame(df.drop(['latitude', 'longitude'], axis=1),
crs={'init': 'epsg:4326'},
geometry=[shapely.geometry.Point(xy)
for xy in zip(df.longitude, df.latitude)])
print(gdf)
world = gpd.read_file(gpd.datasets.get_path('naturalearth_lowres'))
base = world.plot(color='white', edgecolor='black')
gdf.plot(ax=base, marker='o', color='red', markersize=5)
plt.show()
```

OUTPUT:

	City	geometry
0	Berlin	POINT (13.40833 52.51861)
1	Paris	POINT (2.35167 48.85667)
2	Munich	POINT (11.57556 48.13722)



RESULT:

Build cartographic visualization for multiple datasets involving various countries of the world; has been visualized successfully.

EX NO: 9	USE A CASE STUDY ON A DATA SET AND VISUALIZATION TECHNIQUE AND PRESENT ANALYSIS REPORT
DATE:	

Aim:

To use a case study on a data set and apply the various EDA and visualization techniques and present analysis report.

Program:

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
df = pd.read_csv('employees.csv')
df.head()
```

Out[2]:

	First Name	Gender	Start Date	Last Login Time	Salary	Bonus %	Senior Management	Team
0	Douglas	Male	8/6/1993	12:42 PM	97308	6.945	True	Marketing
1	Thomas	Male	3/31/1996	6:53 AM	61933	4.170	True	NaN
2	Maria	Female	4/23/1993	11:17 AM	130590	11.858	False	Finance
3	Jerry	Male	3/4/2005	1:00 PM	138705	9.340	True	Finance
4	Larry	Male	1/24/1998	4:47 PM	101004	1.389	True	Client Services

```
df.shape
```

Out[18]: (1000, 8)

```
df.describe()
```

Out[19]:

	Start Date	Salary	Bonus %
count	1000	1000.000000	1000.000000
mean	1998-12-12 21:50:24	90662.181000	10.207555
min	1980-01-19 00:00:00	35013.000000	1.015000
25%	1990-07-21 12:00:00	62613.000000	5.401750
50%	1999-07-16 00:00:00	90428.000000	9.838500
75%	2007-09-30 12:00:00	118740.250000	14.838000
max	2016-07-15 00:00:00	149908.000000	19.944000
std	NaN	32923.693342	5.528481

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 8 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   First Name         933 non-null    object
 1   Gender             1000 non-null   object
 2   Start Date         1000 non-null   datetime64[ns]
 3   Last Login Time    1000 non-null   object
 4   Salary             1000 non-null   int64
 5   Bonus %            1000 non-null   float64
 6   Senior Management  933 non-null    object
 7   Team               957 non-null    object
dtypes: datetime64[ns](1), float64(1), int64(1), object(5)
memory usage: 62.6+ KB
```

df['Start Date'] = pd.to_datetime(df['Start Date'])

df.nunique()

```
Out[30]: First Name           200
         Gender                3
         Start Date          972
         Last Login Time     720
         Salary              995
         Bonus %             971
         Senior Management     2
         Team                 10
         dtype: int64
```

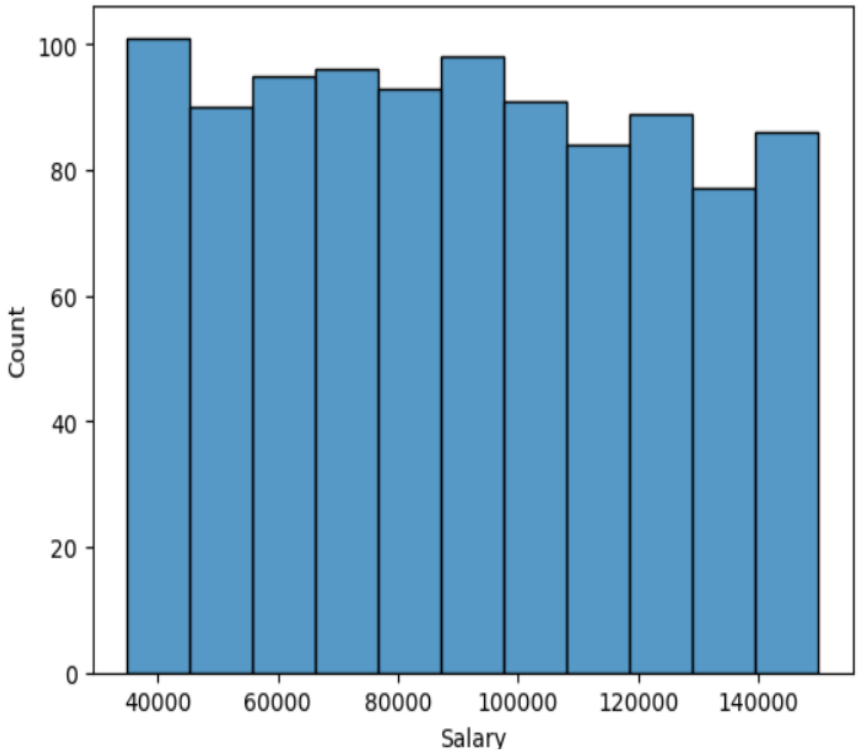
df["Gender"].fillna("No Gender", inplace = True)

df.isnull().sum()

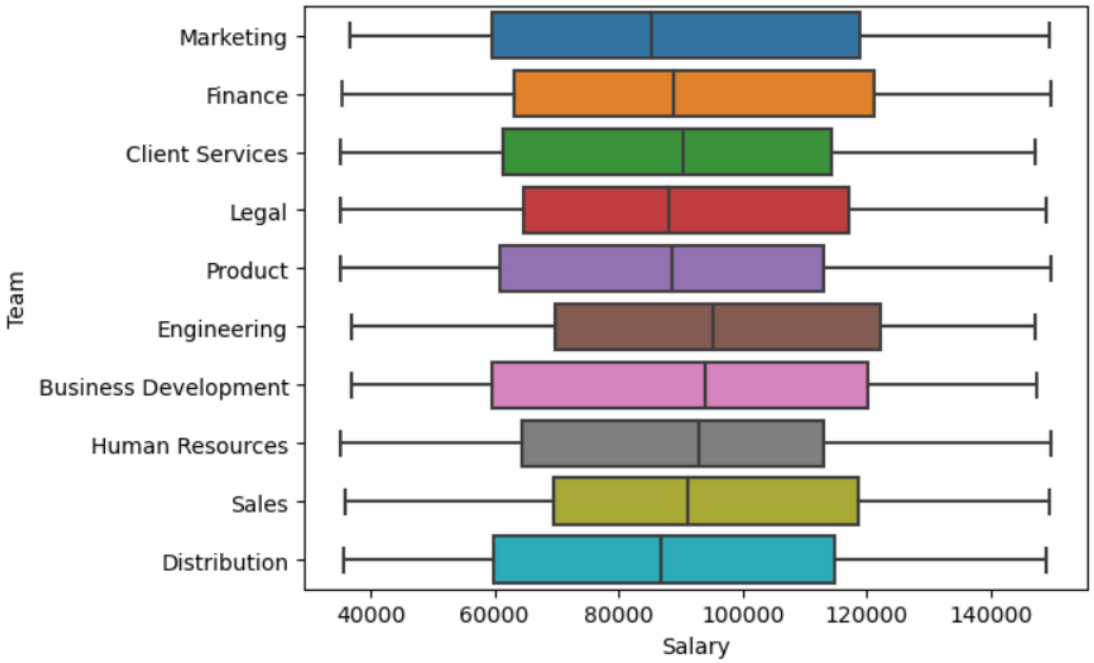
```
Out[26]: First Name           67
         Gender                0
         Start Date            0
         Last Login Time       0
         Salary                0
         Bonus %               0
         Senior Management    67
         Team                 43
         dtype: int64
```

sns.histplot(x='Salary', data=df,)

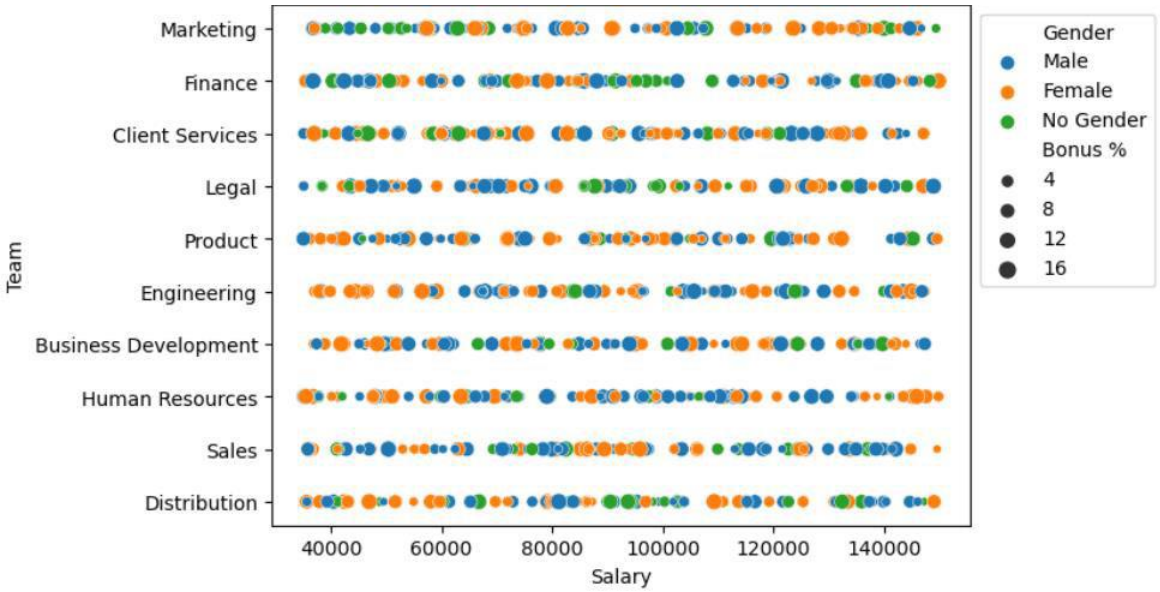
plt.show()



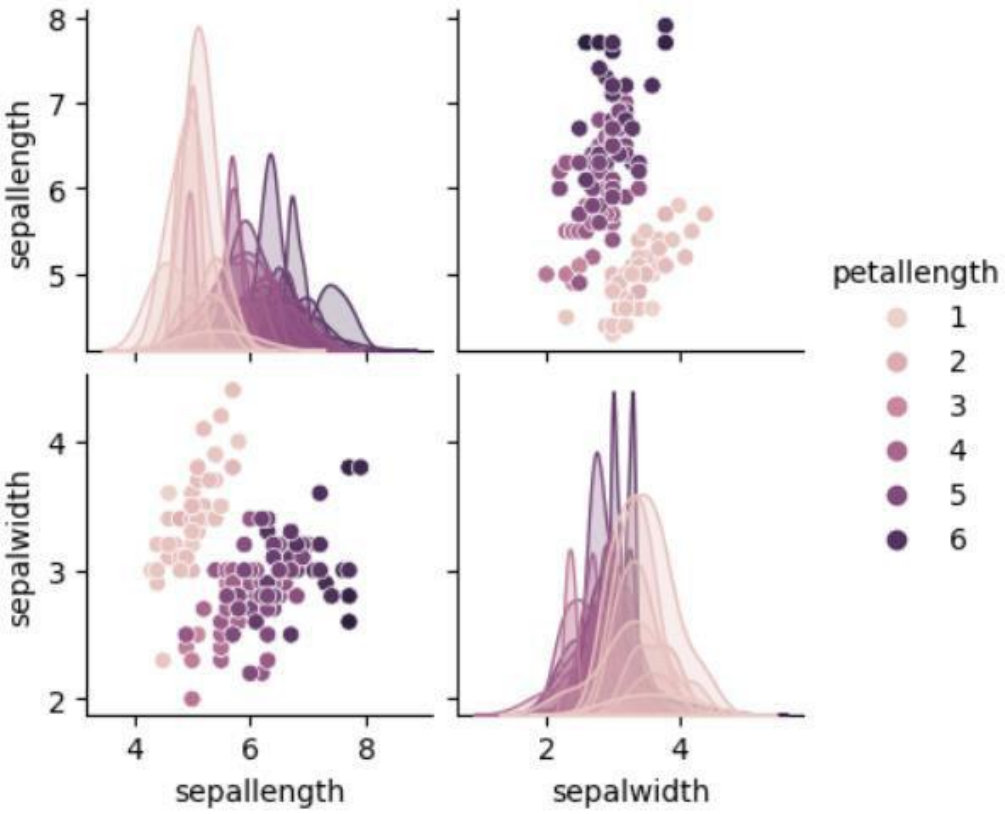

```
sns.boxplot( x="Salary", y='Team', data=df, )
plt.show()
```




```
sns.scatterplot( x="Salary", y='Team', data=df, hue='Gender', size='Bonus %')
plt.legend(bbox_to_anchor=(1, 1), loc=2)
plt.show()
```




```
Out[3]: <seaborn.axisgrid.PairGrid at 0x28295c24950>
```



Result:

Thus EDA and Data Visualization techniques as present an analysis report was done successfully.

Ex No:10	ADVANCED INTERACTIVE DASHBOARD AND REAL-TIME DATA VISUALIZATION
Date:	

Aim:

To create a simple real-time data visualization dashboard using Python Dash and Plotly .

Algorithm:

1. Start the program.
2. Import necessary libraries (Dash, Plotly, Random).
3. Initialize the Dash app.
4. Create empty lists to store X and Y values.
5. Design the layout with a graph and time interval.
6. Write a callback function to update data and refresh the graph automatically.
7. Run the app and view the dashboard in the browser.
8. Stop the program after execution.

Program

```
import random
import time
from collections import deque

import dash
from dash import dcc, html
from dash.dependencies import Input, Output
import plotly.graph_objs as go

# Create the Dash app
app = dash.Dash(__name__)

# We'll keep a rolling window of 20 points
X = deque(maxlen=20)
Y = deque(maxlen=20)
X.append(time.time())
Y.append(random.randint(0, 100))

# Layout (what appears in browser)
```

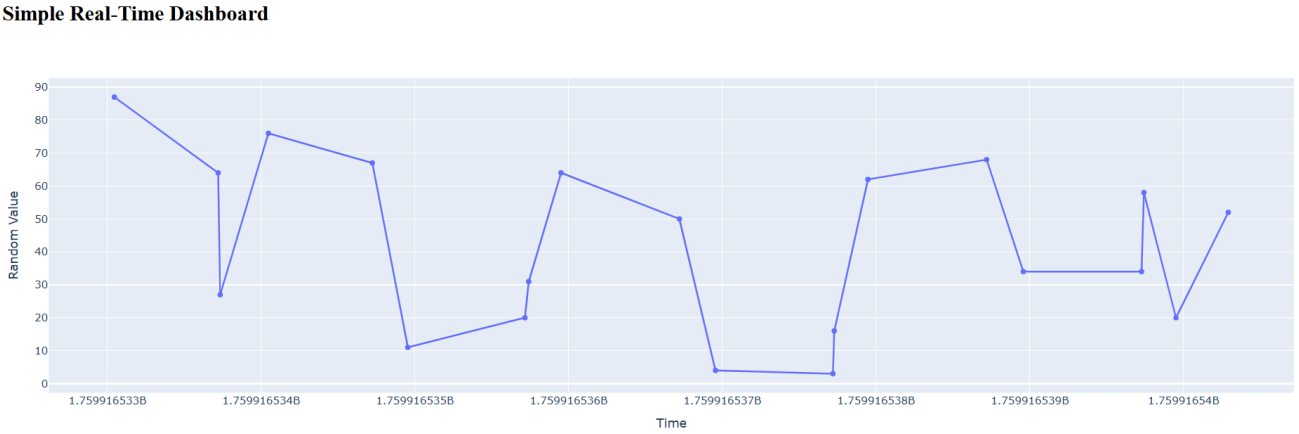
```
app.layout = html.Div([
    html.H2("Simple Real-Time Dashboard"),
    dcc.Graph(id='live-graph', animate=True),
    dcc.Interval(              # updates every 1 second
        id='graph-update',
        interval=1000,         # milliseconds
        n_intervals=0
    )
])

# Callback to update the graph every second
@app.callback(Output('live-graph', 'figure'),
              Input ('graph-update', 'n_intervals'))
def update_graph(n):
    X.append(time.time())
    Y.append(random.randint(0, 100))

    # create a line chart
    fig = go.Figure(
        data=[go.Scatter(x=list(X), y=list(Y), mode='lines+markers')]
    )
    fig.update_layout(
        xaxis_title='Time',
        yaxis_title='Random Value',
        margin=dict(l=40, r=40, t=40, b=40)
    )
    return fig

# Run the app
if __name__ == '__main__':
    app.run(debug=True, port=8050)
```

Output:



Result:

A web-based dashboard showing a line graph that updates every second with a new random value simulating real-time data streaming.